Characterizing Small Circuit Classes from FAC^0 to FAC^1 via Discrete Ordinary Differential Equations

Melissa Antonelli
HIIT & University of Helsinki, Finland

Arnaud Durand
Université Paris Cité, 8, place Aurélie Nemours, Paris, France

Juha Kontinen
University of Helsinki, Helsinki, Finland

Abstract

In this paper, we provide a uniform framework for investigating small circuit classes and bounds through the lens of ordinary differential equations (ODEs). Following an approach recently introduced to capture the class of polynomial-time computable functions via ODE-based recursion schemas and later applied to the context of functions computed by unbounded fan-in circuits of constant depth ($\mathbf{FAC}^0$), we study multiple relevant small circuit classes. In particular, we show that *natural* restrictions on linearity and derivation along functions with specific growth rate correspond to kinds of functions that can be proved to be in various classes, ranging from $\mathbf{FAC}^0$ to $\mathbf{FAC}^1$. This reveals an intriguing link between constraints over linear-length ODEs and circuit computation, providing new tools to tackle the complex challenge of establishing bounds for classes in the circuit hierarchies and possibly enhancing our understanding of the role of counters in this setting. Additionally, we establish several completeness results, in particular obtaining the first ODE-based characterizations for the classes of functions computable in constant depth with unbounded fan-in and Mod 2 gates ($\mathbf{FACC}[\mathbf{2}]$) and in logarithmic depth with bounded fan-in Boolean gates ($\mathbf{FNC}^1$).

2012 ACM Subject Classification Theory of computation → Computational complexity and cryptography; Theory of computation → Circuit complexity

Keywords and phrases Implicit computational complexity, parallel computation, function algebras, ordinary differential equations, circuit complexity

Digital Object Identifier 10.4230/LIPIcs.MFCS.2025.10

Related Version *Full Version*: `https://arxiv.org/abs/2506.23404`

Funding *Melissa Antonelli*: Supported by the Helsinki Institute of Information Technology (HIIT) and the Magnus Ehrnrooth Foundation.
Arnaud Durand: Supported by the Magnus Ehrnrooth Foundation.
Juha Kontinen: Supported by the Magnus Ehrnrooth Foundation.

1 Introduction

Implicit computational complexity is a active area of theoretical computer science that aims to provide machine-independent characterizations of relevant complexity classes. In the context of recursion theory, a foundational result is by Cobham [18], who, in the 1960s, provided an implicit characterization of the class of polynomial-time computable functions ($\mathbf{FP}$) through a *limited* recursion schema. This work not only led to the development of function algebras characterizing multiple classes different from poly-time (see [15]), but also inspired several alternative recursion-theoretic approaches [6, 28, 30]. Among them, the one introduced in [10] shows that ordinary differential equations (ODEs) provide a natural tool for algorithmic design and offers an original characterization of $\mathbf{FP}$, using *linear* systems of equations and *derivation along a logarithmically growing function* (such as the length function ℓ). Informally, the latter condition controls the number of recursion steps, while


© Melissa Antonelli, Arnaud Durand, and Juha Kontinen;
licensed under Creative Commons License CC-BY 4.0

linearity governs the growth of objects generated during the computation. This approach has recently been generalized to the continuous setting [7, 8] and to the study of two small circuit classes [3]. In this paper, we extend the latter far further by presenting a *comprehensive* investigation of circuit classes from $\mathbf{FAC}^0$ to $\mathbf{FAC}^1$ through the lens of ODE schemas.

Although small circuit classes have been characterized in different ways, investigating them through the ODE framework has never been attempted (except for [3]). This approach is both *interesting*, as for a descriptive method based on ODEs to be meaningful and fruitful, it must be able to adjust with very subtle changes in restricted models of computation, and *challenging*, because even simple and useful mathematical functions may not be computable within the considered classes and, consequently, the tools available may seem drastically restricted and the naturalness of the approach questionable. However, the results of this paper show that this is not hopeless. Very simple and natural restrictions on linearity – namely, constraining the values of coefficients and allowing (or not) the call for the sign of the defined function inside the linear term – and on the function along which derivation is performed, allows us to capture the very essence of small circuit computation. Many classical functions – such as polylog bit sum, parity, majority, iterated sum and product, and binary search – are shown to be naturally encapsulated in very simple linear ODE families. For this reason, we believe that our approach to this line of research, which is still in its beginning, can shed some new light on circuit computation and offer a simple (and machine-independent) framework for analyzing these notoriously hard problems through very *natural* and *uniform* constraints on linear ODEs.

Concretely, the results of this paper have led to completely original ODE-based characterizations for the classes of functions computable by unbounded fan-in circuits of constant depth and polynomial size that allow Mod 2 gates ($\mathbf{FACC}[\mathbf{2}]$) and by bounded fan-in circuits of logarithmic depth and polynomial size ($\mathbf{FNC}^1$). We have also improved our understanding of schemas capturing the classes of functions computable by unbounded fan-in circuits of constant depth and polynomial size ($\mathbf{FAC}^0$), possibly including Maj gates ($\mathbf{FTC}^0$), by generalizing and making them part of a more general picture. Finally, we have shown that considering schemas with *derivative along* $\ell_2 = \ell \circ \ell$ is definitely worth investigating: in particular, the corresponding linear schema is in $\mathbf{FNC}^1$, somewhat mirroring what ℓ-ODE does for $\mathbf{FP}$.

Structure of the paper. In Section 2, we present the basics of the method developed in [10], the necessary notions to formulate our results, and recap definitions of the small circuit classes we will consider throughout the paper. In Section 3, we present, in a high-level yet systematic and uniform manner, the main schemas (differently) corresponding to the classes that will be studied. In Section 4, we introduce ODE-based schemas obtained by deriving along ℓ and by imposing different constraints on linearity, and we show that they are computable in the respective circuit classes. We also provide improved completeness results for $\mathbf{FAC}^0$ and $\mathbf{FTC}^0$, along with completely original ODE-based characterizations for $\mathbf{FACC}[\mathbf{2}]$ and $\mathbf{FNC}^1$. In Section 5, we introduce a new class of ODE schemas, the defining feature of which consists of deriving along ℓ_2. Our main result here is the introduction of a very natural linear ℓ_2-ODE schema, the computation of which can be performed in $\mathbf{FNC}^1$. We conclude in Section 6 pointing to possible directions for future research. For space reasons, most proofs are only sketched or omitted in the paper, but can be found in [2]. To make the exposition shorter, most of our characterizations are proved indirectly, as these proofs tend to be more compact. However, direct proofs, both in the uniform and non-uniform settings, can be provided (but are not presented here).

2 Preliminaries

We assume the reader familiar with the basics of complexity theory. In this section, we briefly introduce the approach to complexity delineated in [11] (see the original papers [10, 11] for full details) and a few notions needed for our new characterizations. We also cursorily recall the definitions of the circuit classes that we will consider in the remainder of the paper.

2.1 Capturing Complexity via ODEs

One of the main approaches developed in implicit computational complexity is that based on recursion. It was initiated by Cobham [18], who gave the first characterization for **FP** via a limited recursion schema. This groundbreaking result has inspired not only similar characterizations for other classes, but also alternative recursion-based approaches to capture poly-time computable functions, such as safe recursion [6] and ramification [28, 29]. Among them, the proposal by [10] offers the advantages of not imposing any explicit bound on the recursion schema or assigning specific role to variables. Instead, it is based on Discrete Ordinary Differential Equations (a.k.a. Difference Equations) and combines two novel features, namely derivation along specific functions, to control the number of computation steps, and a special syntactic form of the equation (linearity), to control the object size.

Recall that the *discrete derivative of* $\mathbf{f}(x)$ is defined as $\Delta\mathbf{f}(x) = \mathbf{f}(x+1) - \mathbf{f}(x)$, and that ODEs are expressions of the form

$$\frac{\partial \mathbf{f}(x,\mathbf{y})}{\partial x} = \mathbf{h}\big(x,\mathbf{y},\mathbf{f}(x,\mathbf{y})\big)$$

where $\frac{\partial \mathbf{f}(x,\mathbf{y})}{\partial x}$ stands for the derivative of $\mathbf{f}(x,\mathbf{y})$ considered as a function of x, for a fixed value for $\mathbf{y}$. When some initial value $\mathbf{f}(0,\mathbf{y}) = \mathbf{g}(\mathbf{y})$ is added, this is called Initial Value Problem (IVP). Then, in order to deal with complexity, some restrictions are needed. First, we introduce the notion of *derivation along functions*.

▶ **Notation 1.** For $x \neq 0$, let $\ell(x)$ denote the length function, returning the length of x written in binary, i.e. $\lceil \log_2(x+1) \rceil$, and such that $\ell(0) = 0$. Additionally, let $\ell_2 = \ell \circ \ell$. For $u \geq 0$, let $\alpha(u) = 2^u - 1$ denote the greatest integer whose length is u; similarly, $\alpha_2(u) = 2^{2^u - 1} - 1$ is the greatest integer t such that $\ell_2(t) = u$. Finally, let $\div 2$ denote integer division by 2, i.e. for all $x \in \mathbb{Z}, x \div 2 = \lfloor \frac{x}{2} \rfloor$.

Notice that the value of $\ell(x)$ changes (it increases by 1) when x goes from $2^t - 1$ to 2^t (for $t \geq 1$). Similarly, the value of $\ell_2(x)$ changes when x goes from $2^{2^1 - 1} - 1$ to $2^{2^t - 1}$ (for $t \geq 1$).

▶ **Definition 2** (ℓ-ODE Schema). *Let* $\mathbf{f}, \lambda : \mathbb{N}^p \times \mathbb{Z} \to \mathbb{Z}$ *and* $\mathbf{h} : \mathbb{N}^{p+1} \to \mathbb{Z}$ *be functions. Then,*

$$\frac{\partial \mathbf{f}(x,\mathbf{y})}{\partial \lambda} = \frac{\partial \mathbf{f}(x,\mathbf{y})}{\partial \lambda(x,\mathbf{y})} = \mathbf{h}\big(x,\mathbf{y},\mathbf{f}(x,\mathbf{y})\big) \tag{1}$$

is a formal synonym of $\mathbf{f}(x+1,\mathbf{y}) = \mathbf{f}(x,\mathbf{y}) + \big(\lambda(x+1,\mathbf{y}) - \lambda(x,\mathbf{y})\big) \times \mathbf{h}\big(x,\mathbf{y},\mathbf{f}(x,\mathbf{y})\big)$. *When* $\lambda(x,\mathbf{y}) = \ell(x)$, *we call (1) a* length-ODE *or* ℓ*-ODE.*

Intuitively, one of the key properties of the λ-ODE schema is its dependence on the number of distinct values of λ, i.e., the value of $\mathbf{f}(x,\mathbf{y})$ changes only when the value of $\lambda(x,\mathbf{y})$ does. Computation of solutions of λ-ODEs have been fully described in [11]. Here, we focus on the special case of $\lambda = \ell$, which is particularly relevant for our characterizations. If $\mathbf{f}$ is a solution

of (1) with $\lambda = \ell$ and initial value $\mathbf{f}(0, \mathbf{y}) = \mathbf{g}(\mathbf{y})$, then $\mathbf{f}(1, \mathbf{y}) = \mathbf{f}(0, \mathbf{y}) + \mathbf{h}\big(\alpha(0), \mathbf{y}, \mathbf{f}(\alpha(0), \mathbf{y})\big)$. More generally, for all x and $\mathbf{y}$: $\mathbf{f}(x, \mathbf{y}) = \mathbf{f}(x-1, \mathbf{y}) + \Delta\big(\ell(x-1)\big) \times \mathbf{h}\big(x-1, \mathbf{y}, \mathbf{f}(x-1, \mathbf{y})\big) = \mathbf{f}\big(\alpha(\ell(x)-1), \mathbf{y}\big) + \mathbf{h}\big(\alpha(\ell(x)-1), \mathbf{y}, \mathbf{f}(\alpha(\ell(x)-1), \mathbf{y})\big)$, where $\Delta(\ell(t-1)) = \ell(t) - \ell(t-1)$. Starting from $t = x \geq 1$ and taking successive decreasing values of t, the first difference such that $\Delta(t) \neq 0$ is given by the biggest $t-1$ such that $\ell(t-1) = \ell(x) - 1$, i.e. $t - 1 = \alpha(\ell(x) - 1)$. Setting $\mathbf{h}(\alpha(-1), \mathbf{y}, \cdot) = \mathbf{f}(0, \mathbf{y})$, it follows by induction that:

$$\mathbf{f}(x, \mathbf{y}) = \sum_{u=-1}^{\ell(x)-1} \mathbf{h}\big(\alpha(u), \mathbf{y}, \mathbf{f}(\alpha(u), \mathbf{y})\big)$$

In the present paper, both polynomial expressions (see [3]) and limited polynomial expressions are considered. Let $\mathsf{sg} : \mathbb{Z} \to \mathbb{Z}$ be the sign function over $\mathbb{Z}$, taking value 1 for $x > 0$ and 0 otherwise, and $\mathsf{cosg} : \mathbb{Z} \to \mathbb{Z}$ be the cosign function, such that $\mathsf{cosg}(x) = 1 - \mathsf{sg}(x)$.

▶ **Definition 3.** *A* limited *(*sg*-)*polynomial expression *is an expression over the signature* $\{+, -, \div 2\}$ *(and the* sg *function, resp.) on the set of variables/terms* $X = \{x_1, \ldots, x_h\}$ *and integer constants; (*sg*-)*polynomial expressions *are defined analogously over the signature* $\{+, -, \div 2, \times\}$.

To define the notion of degree for polynomial expressions, we follow the generalization to sets of variables introduced in [3].

▶ **Definition 4.** *Given a list of variables* $\mathbf{x} = x_{i_1}, \ldots, x_{i_m}$*, for* $i_1, \ldots, i_m \in \{1, \ldots, h\}$*, the* degree of a set $\mathbf{x}$ in a sg-polynomial expression P, $\deg(\mathbf{x}, P)$*, is inductively defined as follows:*

- $\deg(\mathbf{x}, x_{i_j}) = 1$*, for* $x_{i_j} \in \{x_{i_1}, \ldots, x_{i_m}\}$*, and* $\deg(\mathbf{x}, x_{i_j}) = 0$*, for* $x_{i_j} \notin \{x_{i_1}, \ldots, x_{i_m}\}$
- $\deg(\mathbf{x}, \mathsf{sg}(P)) = \deg(\mathbf{x}, c) = 0$*, for* c *being an integer constant*
- $\deg(\mathbf{x}, P + Q) = \deg(\mathbf{x}, P - Q) = \max\{\deg(\mathbf{x}, P), \deg(\mathbf{x}, Q)\}$
- $\deg(\mathbf{x}, P \times Q) = \deg(\mathbf{x}, P) + \deg(\mathbf{x}, Q)$ *and* $\deg(\mathbf{x}, P \div 2) = \deg(\mathbf{x}, P)$.

A sg-polynomial expression P is said to be *essentially constant in a set of variables* $\mathbf{x}$ when $\deg(\mathbf{x}, P) = 0$ and *essentially linear in* $\mathbf{x}$, when $\deg(\mathbf{x}, P) = 1$.

▶ **Example 5.** Let $\mathbf{x} = \{x_1, x_2, x_3\}$. The expression $P(x_1, x_2, x_3) = 3x_1x_3 + 2x_2x_3$ is essentially linear in x_1, in x_2 and in x_3. It is not essentially linear in $\mathbf{x}$, as $\deg(P, \mathbf{x}) = 2$. The expression $P'(x_1, x_2, x_3) = x_1 \times \mathsf{sg}((x_1 - x_3) \times x_2) + x_2^3$ is essentially linear in x_1, essentially constant in x_3 and not linear in x_2. Clearly, P' is not linear in $\mathbf{x}$.

The concept of *linearity* allows us to control the growth of functions defined by ODEs. In what follows, we will consider functions $\mathbf{f} : \mathbb{N}^{p+1} \to \mathbb{Z}^d$, i.e. vectors of functions $\mathbf{f} = f_1, \ldots, f_d$ from $\mathbb{N}^{p+1}$ to $\mathbb{Z}$, and introduce the linear ℓ-ODE schema.

▶ **Definition 6** (Linear λ-ODE)**.** *Given* $\mathbf{g} : \mathbb{N}^p \to \mathbb{N}^d, \mathbf{h}, \lambda : \mathbb{N}^{p+1} \to \mathbb{Z}$ *and* $\mathbf{u} : \mathbb{Z}^{d+1} \times \mathbb{N}^{p+1} \to \mathbb{Z}^d$*, the function* $\mathbf{f} : \mathbb{N}^{p+1} \to \mathbb{Z}^d$ *is* linear λ-ODE *definable from* $\mathbf{g}, \mathbf{h}$ *and* $\mathbf{u}$ *if it is the solution of the IVP with initial value* $\mathbf{f}(0, \mathbf{y}) = \mathbf{g}(\mathbf{y})$ *and such that:*

$$\frac{\partial f(x, \mathbf{y})}{\partial \lambda} = \mathbf{u}\big(\mathbf{f}(x, \mathbf{y}), \mathbf{h}(x, \mathbf{y}), x, \mathbf{y}\big)$$

where $\mathbf{u}$ *is* essentially linear in the list of terms $\mathbf{f}(x, \mathbf{y})$*. For* $\lambda = \ell$*, such schema is called* linear length-ODE*.*

If $\mathbf{u}$ is *essentially linear in* $\mathbf{f}(x, \mathbf{y})$, there exist matrices $\mathbf{A}$ and $\mathbf{B}$, whose coefficients are essentially constant in $\mathbf{f}(x, \mathbf{y})$, such that $\mathbf{f}(0, \mathbf{y}) = \mathbf{g}(\mathbf{y})$ and:

$$\frac{\partial \mathbf{f}(x, \mathbf{y})}{\partial \ell} = \mathbf{A}\big(x, \mathbf{y}, \mathbf{h}(x, \mathbf{y}), \mathbf{f}(x, \mathbf{y})\big) \times \mathbf{f}(x, \mathbf{y}) + \mathbf{B}\big(x, \mathbf{y}, \mathbf{h}(x, \mathbf{y}), \mathbf{f}(x, \mathbf{y})\big).$$

Then, for all x and $\mathbf{y}$, $\mathbf{f}(x,\mathbf{y})$ is

$$\sum_{u=-1}^{\ell(x)-1}\left(\prod_{t=u+1}^{\ell(x)-1}\Big(1+\mathbf{A}\big(\alpha(t),\mathbf{y},\mathbf{h}(\alpha(t),\mathbf{y}),\mathbf{f}(\alpha(t),\mathbf{y})\big)\Big)\right)\times\mathbf{B}\big(\alpha(u),\mathbf{y},\mathbf{h}(\alpha(u),\mathbf{y}),\mathbf{f}(\alpha(u),\mathbf{y})\big)$$

with the convention that $\prod_x^{x-1}\kappa(x)=1$ and $(\alpha(-1),\mathbf{y},\cdot,\cdot)=\mathbf{f}(0,\mathbf{y})$. Actually, the use of matrices is not necessary for the scope of this paper, but it is consistent with the general presentation of computation through essentially linear ODEs, of which full details can be found in [10, 11].

One of the main results of [10] is the implicit characterization of **FP** by the algebra made of basic functions $0,1,\pi_i^p,\ell,+,-,\times,\mathsf{sg}$ (where π_i^p denotes the projection function) and closed under composition ($\circ$) and ℓ-ODE.

2.2 On Small Circuit Classes and Function Algebras

Boolean circuits are vertex-labeled directed acyclic graphs whose nodes/gates are either input nodes, output nodes or are labeled with a Boolean function from the set $\{\wedge,\vee,\neg\}$. Boolean circuit with modulo gates allows in addition gates labeled Mod 2 or, more generally, Mod p, that output the sum of the inputs modulo 2 or p, respectively. Finally, Boolean circuit with majority gates are unbounded fan-in circuits including Maj gates, that output 1 when the majority of the inputs are 1's. A family of circuits is **Dlogtime**-uniform if there is a Turing machine (with a random access tape) that decides in deterministic logarithmic time the direct connection language of the circuit, i.e. which, given $1^n,a,b$ and $t\in\{\wedge,\vee,\neg\}$, decides if a is of type t and b is a predecessor of a in the circuit (and analogously for input/output nodes). When dealing with circuits, the resources of interests are *size*, i.e. the number of gates, and *depth*, i.e. the length of the longest path from input to output (see [34] for further details and related results).

▶ **Definition 7** (Classes $\mathbf{FAC}^i$ and $\mathbf{FNC}^i$). *For $i\in\mathbb{N}$, $\mathbf{AC}^i$ (resp., $\mathbf{NC}^i$) is the class of language recognized by a* **Dlogtime***-uniform family of Boolean circuits of unbounded (resp. bounded) fan-in gates of polynomial size and depth $O((\log n)^i)$. We denote by $\mathbf{FAC}^i$ (resp., $\mathbf{FNC}^i$) the corresponding function class.*

It is known that unbounded (poly(n)) fan-in can be simulated using a bounded tree of depth $O(\log n)$, so that $\mathbf{FNC}^i\subseteq\mathbf{FAC}^i\subseteq\mathbf{FNC}^{i+1}$. However, the inclusion has been proved to be strict only for $i=0$. In contrast, when dealing with classes with counting ability, interesting bounds have been established.

▶ **Definition 8** (Classes $\mathbf{FACC}[\mathbf{p}]$ and $\mathbf{FTC}^0$). *The class $\mathbf{ACC}[\mathbf{p}]$ (resp., $\mathbf{TC}^0$) is the class of languages recognized by a* **Dlogtime***-uniform family of Boolean circuits of polynomial size and constant depth including Mod p (resp., Maj) gates . We denote by $\mathbf{FACC}[\mathbf{p}]$ (resp., $\mathbf{FTC}^0$) the corresponding function class.*

We can even consider other levels in the **FTC** hierarchy, such that for $i\in\mathbb{N}$, $\mathbf{FTC}^i$ corresponds to classes of functions computed by families of unbounded fan-in circuits with Maj gates, of polynomial size and depth $O((\log n)^i)$. It is known that the inclusion $\mathbf{FAC}^0\subset\mathbf{FACC}[\mathbf{2}]$ is proper, as the parity function cannot be computed in constant depth by standard unbounded fan-in Boolean gates, but it can be if Mod 2 gates are added. Moreover, since any Mod p gate can be expressed by Maj, $\mathbf{FACC}[\mathbf{p}]\subset\mathbf{FTC}^0$.

As mentioned, Cobham's work paved the way to several recursion-theoretic characterizations for classes other than **FP**, including small circuit ones. In 1988/90, Clote introduced the algebra $\mathcal{A}_0$ to capture functions in the log-time hierarchy (equivalent to $\mathbf{FAC}^0$) using

so-called *concatenation recursion on notation* (CRN) [12, 13]. The greater classes $\mathbf{FAC}^i$ and $\mathbf{FNC}$ are captured adding *weak bounded recursion on notation* [13], while $\mathbf{FNC}^1$ has been characterized both in terms of the *k-bounded recursion on notation* (k-BRN) schema [14] and relying on the basic function *tree* [13]. In 1990, an alternative algebra (and logic), based on a schema called *upward tree recursion*, was introduced to characterize $\mathbf{FNC}^1$ [19]. In 1991, a new recursion-theoretic characterization (and arithmetic *à la Buss*) for $\mathbf{FNC}$ was presented [1]. In 1992 and 1995, other small circuit classes, including $\mathbf{FTC}^0$, were considered (and re-considered) by Clote and Takeuti, who also introduced bounded theories for them [16, 17]. More recently, characterizations for $\mathbf{FNC}^i$ were developed in [9], this time following the ramification approach. In [3], the first ODE-based characterizations for $\mathbf{FAC}^0$ and $\mathbf{FTC}^0$ were introduced. Related approaches to capture small circuit classes have also been provided in the framework of model- and proof-theory, both in the first and second order [26, 5, 24, 31, 19, 1, 16, 17, 27, 21, 4, 20, 22]. To the best of our knowledge, so far, no *uniform* implicit framework to characterize these classes has been offered in the literature.

3 Towards a Uniform Understanding of Small Circuit Classes

We informally outline here our comprehensive investigation of small circuit classes in terms of ODEs (see Figure 1 for a complete description). By slightly modifying the restrictions on the linearity allowed in ℓ-ODE and ℓ_2-ODE, we can perform computation and, in several cases, characterize multiple function classes in the range from $\mathbf{FAC}^0$ to $\mathbf{FAC}^1$.

▶ **Notation 9.** An equation or, by extension, an ODE schema is said to be *strict* when it does not include any call to $f(x, \mathbf{y})$ in A or B (not even under the scope of the sign function). We say that B includes *simple* calls to $f(x, \mathbf{y})$ if $f(x, \mathbf{y})$ occurs in B only in expressions of the form $s = \mathsf{sg}(f(x, \mathbf{y}))$.
For clarity's sake, we will use $k(x, \mathbf{y})$ for functions with limited range, typically in $\{0, 1\}$, and $K(x, \mathbf{y}, h(x, \mathbf{y}), f(x, \mathbf{y}))$ for (possibly limited) sg-polynomial expressions of restricted range. We use $s\big(x, \mathbf{y}, h(x, \mathbf{y}), f(x, \mathbf{y})\big)$ or (when not ambiguous) simply $s(x)$ as an abbreviation for signed expressions in which $f(x, \mathbf{y})$ can occur, i.e. $s = \mathsf{sg}\big(E(x, \mathbf{y}, h(x, \mathbf{y}), f(x, \mathbf{y}))\big)$, where E is any polynomial expression in which $x, \mathbf{y}, h(x, \mathbf{y})$ and $f(x, \mathbf{y})$ may appear. Finally, for readability, we often use

- $s(t)$ as a shorthand for $s\big(\alpha(t), \mathbf{y}, h(\alpha(t), \mathbf{y}), f(\alpha(t), \mathbf{y})\big)$,
- $K(t, \mathbf{y}, h, f)$, or even $K(t)$, as a shorthand for $K(\alpha(t), \mathbf{y}, h(\alpha(t), \mathbf{y}), f(\alpha(t), \mathbf{y}))$,
- $K_i(t)$ (or $B_i(t)$), with $i \in \{0, 1\}$, as a shorthand for $K(\alpha(t), \mathbf{y}, h(\alpha(t), \mathbf{y}), f(\alpha(t), \mathbf{y}))$ ($B(\alpha(t), \mathbf{y}, h(\alpha(t), \mathbf{y}), f(\alpha(t), \mathbf{y}))$, resp.) in which $s(t) = i$.

When different signed expressions $s_1, \ldots, s_r$ occur in A or B these notational conventions are generalized accordingly. If not stated otherwise, we assume that expressions used in $\mathbf{FAC}^0$ and $\mathbf{FACC}[\mathbf{2}]$ schemas are limited sg-polynomial expressions (since multiplication is not in these classes), while those in $\mathbf{FNC}^1$ and $\mathbf{FAC}^1$ are sg-polynomial expressions.

Let us first examine the evolution in the expressive power of the linear schema (see Definition 6), which captures $\mathbf{FP}$,

$$A(x, \mathbf{y}, h, f) \times f(x, \mathbf{y}) + B(x, \mathbf{y}, h, f)$$

when calls to f are not allowed, i.e. when dealing with equation of the form $A(x, \mathbf{y}, h) \times f(x, \mathbf{y}) + B(x, \mathbf{y}, h)$, but still deriving along ℓ. In this case, f can be expressed as a combination of iterated multiplications and products and is in $\mathbf{FTC}^0$. Interestingly, this schema is also

	strict schemas along ℓ	non-strict schemas along ℓ	schemas along ℓ_2
$\mathbf{FAC}^0$	$(k(x)-1)\times f(x)$ $\pm f(x)+k(x)$	$(K(x,f)-1)\times f(x)$	$B(x)$
$\mathbf{FACC}[\mathbf{2}]$	$-k(x)\times f(x)+k(x)\times k'(x)$	$-f(x)+K(x,f)$	
$\mathbf{FTC}^0$	$A(x)\times f(x)+B(x)$	$K(x,f)$ $K(x,f)\times f(x)$	$B(x,f)$
$\mathbf{FNC}^1$		$f(x)+K(x,f)$ $-f(x)+B(x,f)$	$A(x,f)\times f(x)+B(x,f)$
$\mathbf{FAC}^1$		$B(x,f)$	

The table summarizes the features defining the equations of the main linear schemas in the corresponding small circuit class. For readability, the arguments h and $\mathbf{y}$ are omitted. Schemas in blue are not only computable in the corresponding class, but also provide a characterization for it. Calls in non-strict schemas deriving along ℓ are simple or restricted to $s=\mathsf{sg}(f(x)-c)$, for $c\in\mathbb{N}$, except for the schema in $\mathbf{FACC}[\mathbf{2}]$, and B is assumed to take positive values only.

Figure 1 ODE Schemas and Small Circuit Classes in a Nutshell.

the key ingredient to capture $\mathbf{FTC}^0$ and to provide a corresponding completeness result. On the other hand, if calls to f are allowed (i.e. we are back to the general form), but the derivative is along the "slower" function ℓ_2, then expressivity jumps to $\mathbf{FNC}^1$.

As a second example to focus on the smallest classes, consider schemas of the form $-f(x,\mathbf{y})+B(x,\mathbf{y},h,f)$, for $B\geq 0$, with derivative along ℓ and only *simple* calls to f. We show that such defined functions are in $\mathbf{FNC}^1$. Forcing the second part to be in $\{0,1\}$, i.e. considering equations of the forms $-f(x,\mathbf{y})+K(x,\mathbf{y},h,f)$ and $-f(x,\mathbf{y})+k(x,\mathbf{y},h)$ (that is, in the latter case, calls to f are not allowed) makes the functions respectively in $\mathbf{FACC}[\mathbf{2}]$ and $\mathbf{FAC}^0$. Remarkably, in the first two cases, the mentioned schemas are central to characterize the corresponding classes.

Finally, we examine (strict) equations of the form $B(x,\mathbf{y},h)$ (where B is a limited sg-polynomial expression), this time deriving along ℓ_2. Equations of this form provide the ODE equivalent to log iterated sums and, not surprisingly, define a computation in $\mathbf{FAC}^0$. Generalizing them to the non-strict setting by allowing equations of the form $B(x,\mathbf{y},h,f)$, where calls to f occur, gives a really more tricky schema, that is shown computable in $\mathbf{FTC}^0$.

4 Investigating Circuit Complexity Deriving along ℓ

In this section, we present linear ODE schemas obtained by deriving along ℓ and show that they are computable and, in some cases, capable of characterizing the corresponding class.

4.1 The Class $\mathbf{FAC}^0$

We start by revising the characterization of $\mathbf{FAC}^0$ from [3]. Recall that the function algebra $\mathbb{ACDL}$ capturing $\mathbf{FAC}^0$ is based on the two very restrictive forms of ℓ-ODE's below.

▶ **Definition 10** (Schemas ℓ-ODE$_1$, ℓ-ODE$_3$). *Given $g:\mathbb{N}^p\to\mathbb{N}$ and $k:\mathbb{N}^{p+1}\to\{0,1\}$, the function $f:\mathbb{N}^{p+1}\to\mathbb{N}$ is obtained by ℓ-ODE$_1$ from g, and k, if it is the solution of the IVP defined by $f(0,\mathbf{y})=g(\mathbf{y})$ and*

$$\frac{\partial f(x,\mathbf{y})}{\partial \ell}=f(x,\mathbf{y})+k(x,\mathbf{y}).$$

On the other hand, f is obtained by ℓ-ODE$_3$ from g if it is the solution of the IVP defined by $f(0,\mathbf{y}) = g(\mathbf{y})$ and

$$\frac{\partial f(x,\mathbf{y})}{\partial \ell} = -\left\lceil \frac{f(x,\mathbf{y})}{2} \right\rceil.$$

Relying on them, in [3], the ODE-based function algebra below is introduced:

$$\mathbb{ACDL} = [0, 1, \mathsf{sg}, \ell, +, -, \div 2, \#, \pi_i^p; \circ, \ell\text{-ODE}_1, \ell\text{-ODE}_3]$$

where, as standard, $x\#y = 2^{\ell(x)\times\ell(y)}$. In the same paper, this class is shown to precisely characterize $\mathbf{FAC}^0$, see [3, Cor. 19]. Here, we dramatically simplify the (indirect) completeness proof, by showing that ℓ-ODE$_3$ precisely captures the idea of looking for the value of a bit in an $\ell(x)$ long sequence, while ℓ-ODE$_1$ corresponds to the idea of left-shifting (the binary representation of) a given number, possibly adding 1.

▶ Remark 11 (The Function BIT). Given an input y of length $\ell(y)$, for any $i \in \{0,\dots,\ell(y)\}$, the schema ℓ-ODE$_3$ allows us to compute its ith bit in a direct way. Let us consider an instance of ℓ-ODE$_3$ such that $g(\mathbf{y}) = y$, i.e. defined by $\mathsf{rsh}(0,y) = y$ and

$$\frac{\partial \mathsf{rsh}(x,y)}{\partial \ell} = -\left\lceil \frac{\mathsf{rsh}(x,y)}{2} \right\rceil.$$

Then, the $\ell(x)^{th}$ bit of y is $\mathsf{BIT}(x,y) = \mathsf{rsh}(x,y) - \mathsf{rsh}(x+1,y)\times 2$.

▶ Remark 12 (The CRN Schema). In [13], CRN is defined by $f(0,\mathbf{y}) = g(\mathbf{y})$ and $f(s_i(x),\mathbf{y}) = s_{h_i(x,\mathbf{y})}(f(x,\mathbf{y}))$, for $i \in \{0,1\}$, h_i taking value in $\{0,1\}$ and $s_i(x) = 2x+i$ being the binary successor functions. We can rewrite this schema as an instance of ℓ-ODE$_1$ such that

$$\begin{aligned} f_{crn}(0,y,\mathbf{y}) &= g(\mathbf{y}) \\ \frac{\partial f_{crn}(x,y,\mathbf{y})}{\partial \ell} &= f_{crn}(x,y,\mathbf{y}) + h_{\mathsf{BIT}(\ell(y)-\ell(x)-1,y)}(x,\mathbf{y}) \\ &= f_{crn}(x,y,\mathbf{y}) + h_0(x,\mathbf{y}) \times \mathsf{cosg}\big(\mathsf{BIT}(\ell(y)-\ell(x)-1,y)\big) \\ &\qquad + h_1(x,\mathbf{y}) \times \mathsf{sg}\big(\mathsf{BIT}(\ell(y)-\ell(x)-1,y)\big) \end{aligned}$$

where, by definition, for any t, $h_i(t,\mathbf{y}) \le 1$. Then, $\mathsf{crn}(x,\mathbf{y}) = f_{crn}(x,x,\mathbf{y}) = f(x,\mathbf{y})$.

▶ **Theorem 13** ([3]). $\mathbf{FAC}^0 = \mathbb{ACDL}$.

Proof. ($\supseteq$) By [3, Prop. 12, 17]. ($\subseteq$) The proof is indirect and passes through Clote's algebra

$$\mathcal{A}_0 = [0, \pi_i^p, s_0, s_1, \ell, \mathsf{BIT}, \#; \circ, \mathrm{CRN}],$$

which, in [13], is shown sufficient to characterize $\mathbf{FAC}^0$. Its basic functions 0, $\pi_i^p, \ell, \#$ and composition are by design in $\mathbb{ACDL}$, s_0, s_1 can be rewritten in our setting almost for free, while BIT and CRN can be encoded as in Remarks 11 and 12. ◀

In the context of a broader understanding of the hierarchy between classes, we have also considered non-strict ℓ-ODE schemas that are computable in $\mathbf{FAC}^0$. As anticipated, this investigation demonstrates a strong link between depth constraints over circuits and the syntactical constraints that define the corresponding linear length-ODE. Particularly interesting in this setting is the IVP defined by linear equations in which $A \in \{-1,1\}$ and $B = 0$. As long as $f(x,\mathbf{y})$ occurs in $A(= K-1)$ only in expressions of the form $s = \mathsf{sg}(f(x,\mathbf{y}))$, computation through this schema is proved to be in $\mathbf{FAC}^0$. Remarkably, computing the solution of such system corresponds to performing bounded search.

▶ **Lemma 14.** *Let $g : \mathbb{N}^p \to \mathbb{N}$ and $h : \mathbb{N}^{p+1} \to \mathbb{N}$ be computable in* $\mathbf{FAC}^0$*. Then, $f(x, \mathbf{y})$, defined from g and h as the solution of the IVP made of $f(0, \mathbf{y}) = g(\mathbf{y})$ and*

$$\frac{\partial f(x, \mathbf{y})}{\partial \ell} = \big(K(x, \mathbf{y}, h(x, \mathbf{y}), f(x, \mathbf{y})) - 1\big) \times f(x, \mathbf{y})$$

where $K \in \{0, 1\}$ is a limited sg*-polynomial expression and $f(x, \mathbf{y})$ occurs in it only under the scope of the sign function in expressions of the form $s = \mathsf{sg}(f(x, \mathbf{y}))$, is in* $\mathbf{FAC}^0$*.*

Proof Sketch. By definition of ℓ-ODE, $f(x, \mathbf{y}) = \prod_{u=-1}^{\ell(x)-1} K\big(\alpha(u), \mathbf{y}, h(\alpha(u), \mathbf{y}), f(\alpha(u), \mathbf{y})\big)$, with the convention that $K(\alpha(-1), \cdot) = g(\mathbf{y})$. Clearly, if there is a $t \in \{0, \dots, x\}$ such that $K(t) = 0$, then $f(x, \mathbf{y}) = 0$. Therefore, we compute $f(x, \mathbf{y}) = \prod_{u=-1}^{\ell(x)-1} K_1(u)$ which is either 0, if $g(\mathbf{y}) = 0$ or there is a $K_1(u) = 0$, or $g(\mathbf{y})$, otherwise. By hypothesis, we can compute in $\mathbf{FAC}^0$ both $g(\mathbf{y})$ and, for any $u \in \{0, \dots, \ell(x) - 1\}$, $K_1(u)$. Thus, we conclude in one layer defined by an unbounded fan-in $\wedge$-gate. ◀

This result can actually be strengthened by showing that Lemma 14 holds even when generalized expressions of the form $s = \mathsf{sg}(f(x, \mathbf{y}) - r)$ occur in K.

To conclude the presentation of schemas for $\mathbf{FAC}^0$, let us consider simple cases of bounded search encoded by the aforementioned ODE schemas.

▶ **Example 15** (BSearch)**.** Let $R \subseteq \mathbb{N}^{p+1}$ and h_R be its characteristic function. Then, for all x and $\mathbf{y}$, $(\forall z \leq \ell(x))R(z, \mathbf{y}) = \mathsf{sg}(f(x, \mathbf{y}))$, where f is defined by $f(0, \mathbf{y}) = \mathsf{sg}(h_R(0, \mathbf{y}))$ and $\frac{\partial f(x, \mathbf{y})}{\partial \ell} = (h_R(x, \mathbf{y}) - 1) \times f(x, \mathbf{y})$, which is clearly an (very limited) instance of the schema presented in Lemma 14. Another example of limited search consists in checking whether, at some point, $f(x, \mathbf{y})$ reaches or not a boundary, say r. This can be expressed in a natural way by an instance of the (generalized) non-strict schema (in $\mathbf{FAC}^0$) defined by the equation $\frac{\partial f(x, \mathbf{y})}{\partial \ell} = \big(\mathsf{sg}(f(x, \mathbf{y}) - r) - 1\big) \times f(x, \mathbf{y})$.

4.2 The Class FACC[2]

By weakening the constraints on linearity of the schemas which define $\mathbf{FAC}^0$, we introduce an ℓ-ODE the computation of which is provably in $\mathbf{FACC}[\mathbf{2}]$ and actually characterizes this class. We consider a non-strict schema such that $A = -1$ and $B = K \in \{0, 1\}$. As we will see (Section 4.4), this is a special case of the schema we will introduce to capture $\mathbf{FNC}^1$. Together with the functions and schemas defining $\mathbb{ACDL}$, it is also enough to capture $\mathbf{FACC}[\mathbf{2}]$, thus providing the very first ODE-based characterization for this small circuit class "with counters".

▶ **Definition 16** (Schema ℓ-b_0ODE)**.** *Given $g : \mathbb{N}^p \to \mathbb{N}$ and $h : \mathbb{N}^{p+1} \to \mathbb{N}$, the function $f : \mathbb{N}^{p+1} \to \mathbb{N}$ is defined by ℓ-b_0 ODE if it is the solution of the IVP with $f(0, \mathbf{y}) = g(\mathbf{y})$ and*

$$\frac{\partial f(x, \mathbf{y})}{\partial \ell} = -f(x, \mathbf{y}) + K\big(x, \mathbf{y}, h(x, \mathbf{y}), f(x, \mathbf{y})\big)$$

where $K \in \{0, 1\}$ is a limited sg*-polynomial expression and $f(x, \mathbf{y})$ occurs in K only under the scope of the sign function.*

Notably, this schema allows us to capture the computation performed by Mod 2 gates and to rewrite the parity function in a very natural way.

▶ **Example 17** (The Parity Function)**.** Let z be a string of $\ell(z) = n$ bits. To count the number of 1's in z we consider the following system defined by $\mathsf{par}(0, y) = \mathsf{BIT}(0, y)$ and:

$$\frac{\partial \mathsf{par}(x,y)}{\partial \ell} = -\mathsf{par}(x,y) + \big(\mathsf{sg}(\mathsf{par}(x,y)) \times \mathsf{cosg}(\mathsf{BIT}(\ell(x)+1,y)) + \mathsf{cosg}(\mathsf{par}(x,y)) \times \mathsf{sg}(\mathsf{BIT}(\ell(x)+1,y))\big)$$

which is clearly an instance of ℓ-b$_0$ODE. The parity function is now computed by $\mathsf{par}(z, z)$.

Notice that this example shows that ℓ-b$_0$ODE really increases the power of ℓ-ODE$_1$, as the parity function is known not to be in $\mathbf{FAC}^0$ (see [23]). This also indicates that ℓ-b$_0$ODE is actually a schema capturing the specific computational power corresponding to $\mathbf{FACC}[\mathbf{2}]$. Moreover, it is shown that the desired closure property holds for it.

▶ **Lemma 18.** *If $f(x, \mathbf{y})$ is defined by ℓ-b$_0$ODE from functions in $\mathbf{FACC}[\mathbf{2}]$, then $f(x, \mathbf{y})$ is in $\mathbf{FACC}[\mathbf{2}]$ as well.*

Proof Sketch. For simplicity, let us consider the simple case of $f(x, \mathbf{y})$ occurring only under the scope of the sign function in expressions of the form $s = \mathsf{sg}(f(x, \mathbf{y}))$. Relying on the fact that $K(x, \mathbf{y}, h(x, \mathbf{y}), f(x, \mathbf{y})) = K_1(x) \times \mathsf{sg}(f(x, \mathbf{y})) + K_0(x) \times \mathsf{cosg}(f(x, \mathbf{y}))$, we can rewrite the equation defining ℓ-b$_0$ODE as follows:

$$\frac{\partial f(x,\mathbf{y})}{\partial \ell} = -f(x,\mathbf{y}) \times \big(K_0(x) = K_1(x)\big) + \big(K_0(x) = K_1(x)\big) \times \Big(\big((K_0(x) = 1) \times \textsc{Mod}\ 2\Big(\sum_{i=x}^{\ell(x)} \big(K_0(i) = K_0(i+1)\big)\Big) = 0\big) + \big((K_0(x) = 0) \times \textsc{Mod}\ 2\Big(\sum_{i=x}^{\ell(x)} \big(K_0(i) = K_0(i+1)\big)\Big) = 1\big)\Big).$$

Intuitively, if $K_0(t) \neq K_1(t)$, no change occurs. In contrast, if $K_0(t) = K_1(t)$ we restart the computation (as the preceding value of the input must not be the last one for which $K_0 = K_1$) by removing the previous value of $f(t, \mathbf{y})$, and return 1 if either one of the following holds:

- **i** $K_0(t) = K_1(t) = 1$, i.e. $K(t)$ "generates" a 1, and the number of changes of values (temporarily assuming that t is the last value in $\{0, \ldots, \ell(x)-1\}$ such that $K_0(t) = K_1(t)$) between subsequent values, i.e. $K_0(t)$ and $K_0(t+1)$, is such that 1 (possibly alternating) "propagates" until the end of the computation or
- **ii** $K_0(t) = K_1(t) = 0$, i.e. $K(t)$ "generates" a 0, and the subsequent number of changes of values is such that 0 does not propagate to the final value, i.e. $\textsc{Mod}\ 2\Big(\sum_{i=t}^{\ell(x)-1} \big(K_0(t) = K_0(t+1)\big)\Big) = 1$.

It is easy to see that the value computed in this way corresponds to the final value of the computation, i.e. $f(x, \mathbf{y})$ is given by $f(t, \mathbf{y})$ such that for any $t' \in \{t, \ldots, \ell(x)\}, K_0(t') \neq K_1(t')$. Clearly, this is a special case of ℓ-b$_0$ODE in which no call to $f(x, \mathbf{y})$ occurs in A or B, namely of the form $\frac{\partial f(x,\mathbf{y})}{\partial \ell} = -k(x, \mathbf{y}) \times f(x, \mathbf{y}) + k(x, \mathbf{y}) \times k'(x, \mathbf{y})$, for $k(x, \mathbf{y}), k'(x, \mathbf{y}) \in \{0, 1\}$. Computation through this schema can be implemented by circuits of constant depth and polynomial size including $\textsc{Mod}\ 2$ gates: for any $t \in \{0, \ldots, \ell(x) - 1\}$, we can compute $k(\alpha(t), \mathbf{y})$ and $k'(\alpha(t), \mathbf{y})$ in parallel and check both their mutual consistency, i.e. whether $k(\alpha(t), \mathbf{y}) = k'(\alpha(t), \mathbf{y})$, and the consistency between subsequent pairs, i.e. whether $k(\alpha(t), \mathbf{y}) = k(\alpha(t+1), \mathbf{y})$; then, using $\textsc{Mod}\ 2$ gates, we can compute whether the number of changes of values is even or odd and conclude in constant depth via (unbounded fan-in) $\vee$-gate. ◀

Since ℓ-b$_0$ODE has been shown to capture computations performed by Mod 2 gates, our characterization for $\mathbf{FACC}[\mathbf{2}]$ is obtained by simply endowing $\mathbb{ACDL}$ with it.

▶ **Theorem 19.** $\mathbf{FACC}[\mathbf{2}] = [0, 1, \ell, \mathsf{sg}, +, -, \div 2, \#, \pi_i^p; \circ, \ell\text{-ODE}_1, \ell\text{-ODE}_3, \ell\text{-b}_0\text{ODE}]$

Proof. ($\supseteq$) Following the proofs [3, Prop. 12, 17] plus Lemma 18 for ℓ-b$_0$ODE. ($\subseteq$) Consequence of [3, Th. 18] and of the fact that the parity function can be simulated by ℓ-b$_0$ODE (see Example 17). ◀

4.3 The Class $\mathbf{FTC}^0$

In this section, we introduce schemas the computation of which is proved to be in $\mathbf{FTC}^0$. One is defined in terms of strict linear ℓ-ODE and provides a new, compact characterization for this class. The second couple of schemas considered are not strict, as they include simple calls to $f(x, \mathbf{y})$, and are defined uniformly with respect to corresponding (i.e., non-strict) schemas for $\mathbf{FAC}^0$ and $\mathbf{FACC}[\mathbf{2}]$ (as well as for $\mathbf{FNC}^1$ and $\mathbf{FAC}^1$, see Sections 4.4 and 4.5)

We start by dealing with the linear ODE schema defined deriving along ℓ and so that $A(x, \mathbf{y})$ and $B(x, \mathbf{y})$ are both sg-polynomial expressions, with no call to $f(x, \mathbf{y})$.

▶ **Definition 20** (Schema ℓ-pODE). *Given $g : \mathbb{N}^p \to \mathbb{N}$ and $h : \mathbb{N}^{p+1} \to \mathbb{N}$, the function $f : \mathbb{N}^{p+1} \to \mathbb{N}$ is defined by ℓ-pODE from g and h, if it is the solution of the IVP with initial value $f(0, \mathbf{y}) = g(\mathbf{y})$ and such that*

$$\frac{\partial f(x, \mathbf{y})}{\partial \ell} = A\big(x, \mathbf{y}, h(x, \mathbf{y})\big) \times f(x, \mathbf{y}) + B\big(x, \mathbf{y}, h(x, \mathbf{y})\big)$$

where A and B are sg-polynomial expressions.

This compactly encapsulates not only iterated multiplication and addition but also ℓ-ODE$_1$ and ℓ-ODE$_3$ computation.

▶ **Example 21** (Counting and Iterated Addition in ℓ-pODE). Notably, BCount (resp. iterated addition, ItAdd) can be expressed as a special case of ℓ-pODE such that $A = 0$ and $B = k(x, \mathbf{y})$ with $k, g \leq 1$, i.e. by considering an IVP such that $f(0, \mathbf{y}) = g(\mathbf{y})$ and:

$$\frac{\partial f(x, \mathbf{y})}{\partial \ell} = k(x, \mathbf{y}) \qquad (\text{resp. } \frac{\partial f(x, \mathbf{y})}{\partial \ell} = B(x, \mathbf{y}, h(x, \mathbf{y})).$$

Taking $g(\mathbf{y}) = \mathsf{BIT}(0, y)$ and $k(x, \mathbf{y}) = \mathsf{BIT}(\ell(x), y)$, $f(x, x) = \sharp\{u : \mathsf{BIT}(u, x) = 1\}$.

These examples are especially relevant as they reveal that ℓ-pODE is really more expressive than all strict schemas introduced so far (BCount is not computable in $\mathbf{FAC}^0$ and $\mathbf{FACC}[\mathbf{2}]$, see [32, 33]). The closure property can be shown to hold for ℓ-pODE by relying on the following result.

▶ **Proposition 22.** *If $g : \mathbb{N}^p \to \mathbb{N}$ and $h : \mathbb{N}^{p+1} \to \mathbb{N}$ are computable in $\mathbf{FTC}^0$ and $f : \mathbb{N}^{p+1} \to \mathbb{N}$ is defined by ℓ-pODE from g and h, then f is computable in $\mathbf{FTC}^0$ as well.*

Proof. By definition of ℓ-ODE, such f satisfies $f(0, \mathbf{y}) = g(\mathbf{y})$ and

$$f(x, \mathbf{y}) = \sum_{u=-1}^{\ell(x)-1} \left(\prod_{t=u+1}^{\ell(x)-1} \Big(1 + A(r, \mathbf{y})\Big) \right) \times B(u, \mathbf{y})$$

with the convention that $\prod_z^{z-1} \kappa(x) = 1$ and $B(-1, \mathbf{y}) = g(\mathbf{y})$. Computing this expression can be done combining iterated addition and multiplication, which are in $\mathbf{FTC}^0$. ◀

As an immediate byproduct we obtain a new, concise ODE-based characterization of $\mathbf{FTC}^0$.

▶ **Theorem 23.** $\mathbf{FTC}^0 = [0, 1, \ell, \mathsf{sg}, +, -, \div 2, \pi_i^p, \#; \circ, \ell\text{-pODE}]$.

Proof. ($\supseteq$) By Proposition 22. ($\subseteq$) Consequence of the facts that (i) ℓ-pODE is strong enough to express ℓ-ODE$_1$ and ℓ-ODE$_3$, together with $\mathbf{FAC}^0 = \mathcal{A}_0 = \mathbb{ACDL}$ (see [3, 13]), and (ii) ℓ-pODE captures BCount computation (see Example 21), which is equivalent to Maj or $\times$ (see [34]), together with the fact that $[0, \pi_i^p, \mathsf{s}_0, \mathsf{s}_1, \ell, \mathsf{BIT}, \#, \times; \circ, \mathrm{CRN}] = \mathbf{FTC}^0$ (see [17]). established by [17]. ◀

Additionally, in order to better and uniformly understand upper bounds in circuit computation, we consider natural constraints over the non-strict linear length-ODE schema obtained by allowing *simple* calls of $f(x, \mathbf{y})$. As shown throughout the paper, this kind of feature may increase the computational power of a schema, and, here, to ensure that computation remains in $\mathbf{FTC}^0$ we have to restrict the forms of A and B.

▶ **Lemma 24.** *Let $g : \mathbb{N}^p \to \mathbb{N}$ and $h : \mathbb{N}^{p+1} \to \mathbb{N}$ be in $\mathbf{FTC}^0$. Then, a function f defined by the IVP with initial value $f(0, \mathbf{y}) = g(\mathbf{y})$ and equation of one of the following forms:*

$$\frac{\partial f(x, \mathbf{y})}{\partial \ell} = K\big(x, \mathbf{y}, h(x, \mathbf{y}), f(x, \mathbf{y})\big) \text{ or } \frac{\partial f(x, \mathbf{y})}{\partial \ell} = K\big(x, \mathbf{y}, h(x, \mathbf{y}), f(x, \mathbf{y})\big) \times f(x, \mathbf{y})$$

where $K \in \{0, 1\}$ is a sg-polynomial expression and $f(x, \mathbf{y})$ only occurs in expressions of the form $s = \mathsf{sg}(f(x, \mathbf{y}))$ in it, is in $\mathbf{FTC}^0$ as well.

Proof. For the first equation type, notice that, if $g(\mathbf{y}) \geq 1$, then, for any $t \in \{0, \dots, x\}$, $f(t, \mathbf{y}) \geq 0$ (by definition of the schema, in which $A = 0$ and $B \in \{0, 1\}$). Thus, in this case, $f(x, \mathbf{y}) = \sum_{u=-1}^{\ell(x)-1} K_1(u)$ with the convention that $K_1(-1) = g(\mathbf{y})$, and we can conclude by noticing that BCount is in $\mathbf{FTC}^0$. Otherwise, we compute in parallel $g(\mathbf{y})$ and $K_0(t), K_1(t)$ for all $t \in \{0, \dots, \ell(x)\}$. Then, we search for the first t such that $K_0(t) \neq 0$ and compute $\sum_{u=t}^{\ell(x)-1} K_1(u)$: this would be the expected result since, before t, each summand is equal to 0 and, after t, all signed terms remain strictly positive. Again, this corresponds to BCount, which can be done in $\mathbf{FTC}^0$. In terms of circuit computation, this is implemented as follows:

- In parallel, the circuit computes the value of $g(\mathbf{y})$ and, for each $u \in \{0, \dots, \ell(x) - 1\}$ of both $K_0(u)$ and $K_1(u)$ (this can be done in $\mathbf{FTC}^0$ for hypothesis).
- In one layer, for any $t \in \{0, \dots, \ell(x) - 1\}$, it computes the sum $K_0(t) + \sum_{u=t+1}^{\ell(x)-1} K_1(u)$ by t Count gates.
- In constant depth, for any $t \in \{0, \dots, \ell(x) - 1\}$, it computes the conjunction between $\neg K_0(0), \neg K_0(1), ..., \neg K_0(t-1)$ and $K_0(t)$ and select the corresponding Count gate.

For the second equation type, first note that, if $g(\mathbf{y}) = 0$, trivially $f(x, \mathbf{y}) = 0$ (which is clearly computable in $\mathbf{FTC}^0$). Let $g(\mathbf{y}) \geq 1$. Then, for any $t \in \{0, \dots, \ell(x)-1\}$, $\mathsf{sg}(f(0, \mathbf{y})) > 0$. Therefore, by definition of ℓ-ODE, $f(x, \mathbf{y}) = \prod_{u=-1}^{\ell(x)-1} \big(1 + K_1(u)\big)$, with $\prod_{t=-1}^{t'} \kappa = g(\mathbf{y})$. This can be computed in $\mathbf{FTC}^0$: $g(\mathbf{y})$ and each $K_1(u')$, for $u' \in \{0, \dots, \ell(x) - 1\}$ can be computed in parallel in the beginning ($g(x, \mathbf{y})$ and $h(x, \mathbf{y})$ being computable in $\mathbf{FTC}^0$ for hypothesis and K_1 being a sg-polynomial expression by definition) and ItMult (so, in particular, iterated multiplication by 1 and 2) is in $\mathbf{FTC}^0$, see [25]. ◀

4.4 The Class $\mathbf{FNC}^1$

In this section, we consider two restrictions on ℓ-ODE, which give us natural schemas the solution of which is a function computable in $\mathbf{FNC}^1$: namely, the schema defined by $A = 1$ and $B = K \in \{0, 1\}$ and the one defined by $A = -1$ and more liberal $B \geq 0$ (obtaining the so-called ℓ-bODE schema, somewhat generalizing ℓ-b$_0$ODE).

▶ **Lemma 25** (Bounded Concatenation with Simple Calls). *Let $g : \mathbb{N}^p \to \mathbb{N}$ and $h : \mathbb{N}^{p+1} \to \mathbb{N}$ be computable in* $\mathbf{FNC}^1$*. Then, computation through the schema defined by the IVP with initial value $f(0, \mathbf{y}) = g(\mathbf{y})$ and such that:*

$$\frac{\partial f(x, \mathbf{y})}{\partial \ell} = f(x, \mathbf{y}) + K\big(x, \mathbf{y}, h(x, \mathbf{y}), f(x, \mathbf{y})\big)$$

where $K \in \{0, 1\}$ is a sg*-polynomial expression, and $f(x, \mathbf{y})$ appears in it only under the scope of the sign function in expressions of the form $s = \mathsf{sg}(f(x, \mathbf{y}))$, is in* $\mathbf{FNC}^1$*.*

Proof Sketch. By definition of ℓ-ODE, $f(x, \mathbf{y}) = \sum_{u=-1}^{\ell(x)-1} 2^{\ell(x)-u-1} \times K(\alpha(u), \mathbf{y}, h(\alpha(u), \mathbf{y}), f(\alpha(u), \mathbf{y}))$, with the convention that $K(\alpha(-1)) = g(\mathbf{y})$. Let us define the truncated sum (or concatenation) between a and $b-1$ bounded by $\ell(x) - 1$, as $S(a, b) = \sum_{u=a}^{b-1} K\big(\alpha(u), \mathbf{y}, h(\alpha(u), \mathbf{y}), S(u)\big)$, with $S(u) = S(\alpha(u), \mathbf{y}) = f(\alpha(u), \mathbf{y})$. In particular, we consider two sums in which the sign of the first term is assumed to be either 0 or 1: $S_0(a, b) = \sum_{u=a}^{b-1} K(\alpha(u), \mathbf{y}, h(\alpha(u), \mathbf{y}), S(u))$, with $S_0(a) = 0$ and $S_1(a, b) = \sum_{u=a}^{b-1} 2^{\ell(x)-u-1} \times K_1(u)$. Therefore, for any x and $\mathbf{y}$:

$$f(x, \mathbf{y}) = S(-1, \ell(x)/2) + S(\ell(x)/2, \ell(x)) = + \begin{cases} S(-1, \ell(x)/2) \\ S_0(\ell(x)/2, \ell(x)) \times (1 - s(\ell(x)/2 - 1)) \\ S_1(\ell(x)/2, \ell(x)) \times s(\ell(x)/2 - 1). \end{cases}$$

Clearly, $S(-1, \ell(x)/2), S_0(\ell(x)/2, \ell(x))$ and $S_1(\ell(x)/2, \ell(x))$ can be computed in parallel: we know the actual starting value of $S_0(\ell(x)/2, \ell(x))$, which is 0, while to compute $S_1(\ell(x)/2, \ell(x)) = \sum_{u=\ell(x)/2}^{\ell(x)-1} 2^{\ell(x)-u-1} K_1(u)$ it is enough to know that, for any $a > \ell(x)/2$, $f(\alpha(a), \mathbf{y}) > 0$. Once $S(-1, \ell(x)/2) = f(\alpha(\ell(x)/2), \mathbf{y})$ is computed and its sign tested, we can select between S_0 and S_1 accordingly. We conclude by taking the concatenation between $S(-1, \ell(x)/2)$ and either S_0 or S_1. ◀

This schema can be generalized so to allow calls to $f(x, \mathbf{y})$ in expressions of the form $s = \mathsf{sg}(f(x, \mathbf{y}) - c)$, i.e. testing whether $f(x, \mathbf{y}) \geq c$. Computation through an extended version in which $f(x, \mathbf{y})$ occurs under the scope of bit is provably in $\mathbf{FNC}^1$ but it is also strong enough to capture perfect binary tree structures, hence to provide a direct encoding of $\mathbf{FNC}^1$ circuit computation.

▶ Remark 26. The condition that f appears only in expressions of the form $s = \mathsf{sg}(f(x, \mathbf{y}))$ or even $\mathsf{sg}(f(x, \mathbf{y}) - c)$ might seem limited. However, the possibilities it offers really increase the ODE's expressive power and, on the other side, even for slightly more liberal conditions, provide the only known upper bound for the schema is $\mathbf{FP}$.

We now move to the new characterization of $\mathbf{FNC}^1$. To establish it, we introduce an ℓ-ODE schema defined by imposing $A = -1$ and $B \geq 0$, that is by generalizing the constraint defining ℓ-b$_0$ODE from $B = K \in \{0, 1\}$ to B taking any positive value.

▶ **Definition 27** (Schema ℓ-bODE). *Given $g : \mathbb{N}^p \to \mathbb{N}$ and $h : \mathbb{N}^{p+1} \to \mathbb{N}$, the function $f : \mathbb{N}^{p+1} \to \mathbb{N}$ is defined by ℓ-bODE if it is the solution of the IVP with initial value $f(0, \mathbf{y}) = g(\mathbf{y})$ and such that*

$$\frac{\partial f(x, \mathbf{y})}{\partial \ell} = -f(x, \mathbf{y}) + B\big(x, \mathbf{y}, h(x, \mathbf{y}), f(x, \mathbf{y})\big)$$

where, for any $x, \mathbf{y}$, $B(x, \mathbf{y}, h(x, \mathbf{y}), f(x, \mathbf{y})) \geq 0$ is a sg*-polynomial expression, and $f(x, \mathbf{y})$ occurs in it only in expressions of the form $\mathsf{sg}(f(x, \mathbf{y}) - c)$, being c a constant.*

▶ **Lemma 28.** *If $f(x,\mathbf{y})$ is defined by ℓ-bODE from functions in $\mathbf{FNC}^1$, then $f(x,\mathbf{y})$ is in $\mathbf{FNC}^1$ as well.*

Proof. To simplify the exposition, w.l.o.g. let us suppose that: (i) B includes only one signed term, say s, in which $f(x,\mathbf{y})$ occurs, and (ii) $\ell(x)$ is a power of two (to avoid rounding). By definition of ℓ-ODE, $f(x,\mathbf{y}) = B\big(\alpha(\ell(x)-1),\mathbf{y},h(\alpha(\ell(x)-1),\mathbf{y}),f(\alpha(\ell(x)-1),\mathbf{y})\big) = B_i(\ell(x)-1)$, with $i = s(\ell(x)-1)$. We consider partial or truncated results between a and $b-1$ bounded by $\ell(x)-1$, intuitively corresponding to subsequent compositions of s in B, going from a to $b-1$; that is, either $C(a,b) = C_0(a,b)$, i.e. the computation of $f(b-1,\mathbf{y})$ starting from $f(a,\mathbf{y}) = B_0(a-1)$, or $C(a,b) = C_1(a,b)$, i.e. the computation of $f(b-1,\mathbf{y})$ starting from $f(a,\mathbf{y}) = B_1(a-1)$. The values of $C(0,a-1)$, $C_0(a,b)$ and $C_1(a,b)$ can be computed independently from all previous values of $f(\cdot,\mathbf{y})$. Additionally, only one between $C_0(a,b)$ and $C_1(a,b)$ corresponds to the actual value of $C(a,b)$. Then, for any x and $\mathbf{y}$, the value of $f(x,\mathbf{y})$ can be computed inductively by selecting the correct partial $B(a,b)$ as follows: $f(x,\mathbf{y}) = C(-1,\ell(x)) = \mathsf{sg}\big(C(-1,\ell(x)/2)\big)\times C_1\big(\ell(x)/2,\ell(x)\big)+\mathsf{cosg}\big(C(-1,\ell(x)/2)\big)\times C_0\big(\ell(x)/2,\ell(x)\big)$. Both $C(-1,\ell(x)/2), C_0(\ell(x)/2,\ell(x))$ and $C_1(\ell(x)/2,\ell(x))$ as well as, for any $t\in\{0,\dots,\ell(x)-1\}$, $B_0(t), B_1(t)$ and the corresponding signs can be computed independently from each other and in parallel. The number of subsequent recursive calls needed to end up with the desired selection, i.e. the one corresponding to $C_0(\ell(x)/2,\ell(x))$ and $C_1(\ell(x)/2,\ell(x))$, would require $\ell(x)/2$ (selection) steps, ensuring we are in $\mathbf{FNC}^1$. ◀

▶ Remark 29. As desired, Lemma 28 still holds when dealing with $s_0,\dots,s_r$ signed expressions occurring in B. To deal with this more general case we need to add an initial consistency check, considering $\mathsf{sg}(B_{i_0,\dots,i_r}(t))$'s and $B_{i'_0\dots i'_r}(t+1)$'s, in the very beginning.

Together with ℓ-pODE, ℓ-bODE provides a new ODE-based characterization for $\mathbf{FNC}^1$.

▶ **Theorem 30.** $\mathbf{FNC}^1 = [0,1,\ell,\mathsf{sg},+,-,\div 2,\#,\pi_i^p;\circ,\ell\text{-pODE},\ell\text{-bODE}]$.

Proof. ($\supseteq$) All basic functions of this algebra are already in $\mathbf{FAC}^0$ and the closure property holds for $\circ,\ell$-pODE (generalization of Proposition 22) and ℓ-bODE (Lemma 28).
($\subseteq$) The proof is indirect and passes through Clote's algebra:

$$\mathcal{N}'_0 = [0,\pi_i^p,\mathsf{s}_0,\mathsf{s}_1,\ell,\mathsf{BIT},\#;\circ,\mathrm{CRN},4\text{-BRN}],$$

where a function is defined by 4-BRN if $f(0,\mathbf{y}) = g(\mathbf{y})$ and $f(s_i(x),\mathbf{y}) = h_i(x,\mathbf{y},f(x,\mathbf{y}))$ for g and h taking values in $\{0,\dots,4\}$ only. In [14], this class is proved to be the one characterizing $\mathbf{FNC}^1$. In particular, to show that any function defined by 4-BRN can be rewritten as the solution of an instance of ℓ-bODE, we consider the IVP defined by the initial value $\mathsf{4brn}(x,y,\mathbf{y}) = g(\mathbf{y})$ and such that:

$$\begin{aligned}\frac{\partial \mathsf{4brn}(x,y,\mathbf{y})}{\partial \ell} = -\mathsf{4brn}(x,y,\mathbf{y}) &+ h_{z(x)}(\ell(x),\mathbf{y},0)\times\mathsf{cosg}\big(\mathsf{4brn}(x,y,\mathbf{y})\big)\\ &+ h_{z(x)}(\ell(x),\mathbf{y},1)\times\mathsf{sg}\big(\mathsf{4brn}(x,y,\mathbf{y})\big)\times\mathsf{cosg}\big(\mathsf{4brn}(x,y,\mathbf{y})-1\big)\\ &+ h_{z(x)}(\ell(x),\mathbf{y},2)\times\mathsf{sg}\big(\mathsf{4brn}(x,y,\mathbf{y})-1\big)\times\mathsf{cosg}\big(\mathsf{4brn}(x,y,\mathbf{y})-2\big)\\ &+ h_{z(x)}(\ell(x),\mathbf{y},3)\times\mathsf{sg}\big(\mathsf{4brn}(x,y,\mathbf{y})-2\big)\times\mathsf{cosg}\big(\mathsf{4brn}(x,y,\mathbf{y})-3\big)\\ &+ h_{z(x)}(\ell(x),\mathbf{y},4)\times\mathsf{sg}\big(\mathsf{4brn}(x,y,\mathbf{y})-3\big)\times\mathsf{cosg}\big(\mathsf{4brn}(x,y,\mathbf{y})-4\big)\end{aligned}$$

with $z(x) = \mathsf{BIT}(\ell(y)-\ell(x)-1,y)$. Then, $\mathsf{4brn}(x,x,\mathbf{z}) = f(x,x,\mathbf{z})$, where f is defined by 4-BRN from g and h_i. Since Clote's $\mathcal{N}'_0$ is nothing but $\mathcal{A}_0$ endowed with 4-BRN [14] and by [3] $\mathcal{A}_0 = \mathbb{ACDL}$, this encoding of 4-BRN via ℓ-bODE, together with Theorem 23, concludes our proof. ◀

4.5 The Class $\mathbf{FAC}^1$

When we consider a generalization of the non-strict schemas shown in $\mathbf{FTC}^0$ from $B = K \in \{0, 1\}$ to more liberal $B \geq 0$ (analogously to the generalization from ℓ-b$_0$ODE to ℓ-bODE), we obtain an ℓ-ODE schema, which is provably in $\mathbf{FAC}^1$. Formally, it is defined by an IVP with restrictions $A = 0$ and, as mentioned, $B \geq 0$ with only simple calls to $f(x, \mathbf{y})$.

The proof of the result below is similar to that for Lemma 25, except for the last step which amounts of a sum (possibly including carries) rather than a concatenation.

▶ **Lemma 31** (Bounded Sum with Simple Calls). *Let $g : \mathbb{N}^p \to \mathbb{N}$ and $h : \mathbb{N}^{p+1} \to \mathbb{N}$ be computable in $\mathbf{FAC}^1$. Then, computation through the schema defined by the IVP with initial value $f(0, \mathbf{y}) = g(\mathbf{y})$ and such that*

$$\frac{\partial f(x, \mathbf{y})}{\partial \ell} = B\big(x, \mathbf{y}, h(x, \mathbf{y}), f(x, \mathbf{y})\big)$$

where $B \geq 0$ is a sg*-polynomial expression with $f(x, \mathbf{y})$ only appearing under the scope of the sign function inside expressions of the form $s = \mathsf{sg}(f(x, \mathbf{y}))$, is in $\mathbf{FAC}^1$ as well.*

5 Characterizing Classes Deriving along ℓ_2

Here, we consider alternative linear ODEs in which derivation is not along ℓ but along the "slower" function ℓ_2. Of course, in this context, same syntactic constraints on the linearity of the schema end up characterizing smaller classes (for example, the solution of the system shown in $\mathbf{FAC}^1$ when deriving along ℓ is shown to be computable in $\mathbf{FTC}^0$ if derivation is along ℓ_2). The main result of this section is that linear ODEs with derivative along ℓ_2 are in $\mathbf{FNC}^1$ (rather than $\mathbf{FP}$, as in the case of ℓ [10]).

First, observe that when deriving along ℓ_2, computation through a strict linear ODE schema is (of course in $\mathbf{FTC}^0$ but even) in $\mathbf{FAC}^0$ and corresponds to LogItAdd, i.e. iterated addition of ℓ_2 numbers.

▶ **Lemma 32.** *Let $g : \mathbb{N}^p \to \mathbb{N}$ and $h : \mathbb{N}^{p+1} \to \mathbb{N}$ be computable in $\mathbf{FAC}^0$. Then, $f : \mathbb{N}^{p+1} \to \mathbb{N}$ defined from h and g by the IVP with initial value $f(0, \mathbf{y}) = g(\mathbf{y})$ and such that*

$$\frac{\partial f(x, \mathbf{y})}{\partial \ell_2} = B\big(x, \mathbf{y}, h(x, \mathbf{y})\big)$$

where B is a limited sg*-polynomial expression, is in $\mathbf{FAC}^0$.*

When this schema is generalized to the non-strict setting by allowing $f(x, \mathbf{y})$ to occur in B under the scope of the sign function, computation is proved to be in $\mathbf{FTC}^0$ by a much evolved argument.

▶ **Lemma 33.** *Let $g : \mathbb{N}^p \to \mathbb{N}$ and $h : \mathbb{N}^{p+1} \to \mathbb{N}$ be definable in $\mathbf{FTC}^0$. Then, computation through the schema defined by the IVP with initial value $f(0, \mathbf{y}) = g(\mathbf{y})$ and such that*

$$\frac{\partial f(x, \mathbf{y})}{\partial \ell_2} = B\big(x, \mathbf{y}, h(x, \mathbf{y}), f(x, \mathbf{y})\big)$$

where B is a sg*-polynomial expression in which $f(x, \mathbf{y})$ only appears under the scope of the sign function, is in $\mathbf{FTC}^0$.*

As mentioned, the IVP defined deriving along ℓ_2 and allowing "full linearity" is particularly interesting.

▶ **Definition 34** (Schema ℓ_2-ODE). *Given $g : \mathbb{N}^p \to \mathbb{N}$ and $h : \mathbb{N}^{p+1} \to \mathbb{N}$, the function $f : \mathbb{N}^{p+1} \to \mathbb{N}$ is defined by ℓ_2-ODE if it is the solution of the IVP with initial value $f(0, \mathbf{y}) = g(\mathbf{y})$ and such that:*

$$\frac{\partial f(x, \mathbf{y})}{\partial \ell_2} = A\big(x, \mathbf{y}, h(x, \mathbf{y}), f(x, \mathbf{y})\big) \times f(x, \mathbf{y}) + B\big(x, \mathbf{y}, h(x, \mathbf{y}), f(x, \mathbf{y})\big).$$

This schema captures the computation of expressions in the following form:

$$f(x, \mathbf{y}) = \sum_{u=-1}^{\ell_2(x)-1} \left(\prod_{z=u+1}^{\ell_2(x)-1} \Big(1 + A\Big(\alpha_2(z), \mathbf{y}, h(\alpha_2(z), \mathbf{y}), f(\alpha_2(z), \mathbf{y})\Big)\Big) \right) \times B\Big(\alpha_2(u), \mathbf{y}, h(\alpha_2(u), \mathbf{y}), f(\alpha_2(u), \mathbf{y})\Big),$$

where $\alpha_2(v) = 2^{2^v - 1} - 1$, i.e. $\ell_2(\alpha_2(v)) = v$ and $\alpha_2(v)$ is the greatest integer t such that $\ell_2(t) = v$.

▶ **Lemma 35.** *Let f be defined by ℓ_2-ODE from g and h in $\mathbf{FNC}^1$, then f is in $\mathbf{FNC}^1$.*

6 Conclusion

The objective of this paper was to start a new and uniform approach for studying circuit complexity through the lens of ODEs. This approach allows to consider bounds on size or depth in circuit computation in terms of constraints on the linearity of the equations defining recursion schemas in two main contexts of derivatives: derivation along ℓ and along ℓ_2. Additionally, we have investigated completeness both improving and placing in a coherent setting the results obtained in [3] for $\mathbf{FAC}^0$ and $\mathbf{FTC}^0$, and providing what, to the best of our knowledge, are the first ODE-based characterizations for $\mathbf{FACC}[\mathbf{2}]$ and $\mathbf{FNC}^1$. In addition of being a simple and natural way to design algorithms, ODE tools have also enabled us to obtain relatively simple and self-contained membership proofs and characterizations compared to the often convoluted constructions or external theories referenced in previous literature.

Many directions for future investigation remain open. A natural one would be the study of characterizations for the entire **FAC** and **FNC** hierarchies. For this, one has to identify which ODE features or characteristic would help to jump from one depth level, say $(\log n)^i$, to the next. Furthermore, a possible next step of our research involves defining (strict) ℓ-ODE schemas corresponding to complexity classes with counters, lying between $\mathbf{FACC}[\mathbf{2}]$ and $\mathbf{FTC}^0$ (i.e. which correspond to computations executed by unbounded fan-in circuits featuring MOD p gates), ultimately allowing us to better understand their interrelationships and the concept of counting within the framework of small circuits. Another relevant and intriguing direction is the exploration of the expressive power of schemas with derivative along ℓ_2 in particular w.r.t. the problem of capturing complexity classes. Studying the relationship between ℓ and ℓ_2 schemas might also be informative. Classical methods such as changes of variables sometimes help to go from one ODE schema to another but, on the other side, it seems that changing the function along which the derivative is taken gives a different point of view in terms of what can be expressed. However, this study has just been initiated here, and a systematic investigation is left for future work. Other challenging directions for exploration include generalizing our approach to the continuous setting by analyzing small circuit classes over the reals, following the path outlined in [7] for **FP** and **FPSPACE**, and developing proof-theoretical counterparts to our ODE-style algebras, in the form of natural rule systems inspired by the ODE design, to syntactically characterize the corresponding classes.

References

1 B. Allen. Arithmetizing uniform NC. *Ann. Pure Appl. Logic*, 53:1–50, 1991. doi:10.1016/0168-0072(91)90057-S.

2 M. Antonelli, A. Durand, and J. Kontinen. Characterizing small circuit classes from FAC^0 to FAC^1 via discrete ordinary differential equations. URL: https://arxiv.org/abs/2506.23404.

3 M. Antonelli, A. Durand, and J. Kontinen. A new characterization of FAC^0 via discrete ordinary differential equations. In *Proc. MFCS*, 2024.

4 T. Arai. A bounded arithmetic AID for Frege systems. *Ann. Pure Appl. Logic*, 103:155–199, 2000. doi:10.1016/S0168-0072(99)00041-X.

5 D.A.M. Barrington, N. Immerman, and H. Straubing. On uniformity within NC^1. *J. of Comput. and Syst. Sc.*, 41:274–306, 1990. doi:10.1016/0022-0000(90)90022-D.

6 S. Bellantoni and S. Cook. A new recursion-theoretic characterization of poly-time functions. *Comput. Complex.*, 2:97–110, 1992. doi:10.1007/BF01201998.

7 M. Blanc and O. Bournez. A characterisation of functions computable in polynomial time and space over the reals with discrete ordinary differential equations: simulation of Turing Machines with analytic discrete ODEs. In *Proc. MFCS*, pages 21:1–21:15, 2023.

8 M. Blanc and O. Bournez. The complexity of computing in continuous time: Space complexity is precision. In *Proc. ICALP*, pages 129:1–129:22, 2024.

9 G. Bonfante, R. Kahle, J.-Y. Marion, and I. Oitavem. Two function algebras defining functions in NC^k boolean circuits. *Inf. and Comput.*, 2016.

10 O. Bournez and A. Durand. Recursion schemes, discrete differential equations and characterization of polynomial time computation. In *Proc. MFCS*, 2019.

11 O. Bournez and A. Durand. A characterization of functions over the integers computable in polynomial time using discrete differential equations. *Comput. Complex.*, 32(7), 2023. doi:10.1007/S00037-023-00240-1.

12 P.G. Clote. A sequential characterization of the parallel complexity class NC. Technical report, Boston College, 1988.

13 P.G. Clote. *Sequential, machine-independent characterizations of the parallel complexity classes AlogTIME, AC^k, NC^k and NC*, pages 49–69. Progress in Computer Science and Applied Logic. Birkhäuser, Boston, MA, 1990.

14 P.G. Clote. On polynomial size Frege proofs of certain combinatorial principles. In P. Clote and Krjícek J., editors, *Arithmetic, Proof Theory, and Computational Complexity*, pages 166–184. Clarendon Press, 1993.

15 P.G. Clote and E. Kranakis. *Boolean Functions and Computation Models*. Springer, 1998.

16 P.G. Clote and G. Takeuti. Bounded arithmetic for NC, ALogTIME, L and NL. *Ann. Pure and Appl. Logic*, 56:73–117, 1992. doi:10.1016/0168-0072(92)90068-B.

17 P.G. Clote and G. Takeuti. First order bounded arithmetic and small complexity classes. In *Feasible Mathematics II*, pages 154–218. Birkhäuser Boston, 1995.

18 A. Cobham. The intrinsic computational difficulty of functions. In *Logic, Methodology and Phylosophy of Science: Proc. 1964 International Congress*, pages 24–30. North-Holland Pub. Co., 1965.

19 K.J. Compton and C. Laflamme. An algebra and a logic for NC^1. *Inf. Comput.*, 87(1/2):240–262, 1990. doi:10.1016/0890-5401(90)90063-N.

20 S. Cook and T. Morioka. Quantified propositional calculus and a second-order theory for NC^1. *Arch. Math. Logic*, 44:711–749, 2005. doi:10.1007/S00153-005-0282-2.

21 S. Cook and P. Nguyen. Theories for TC^0 and other small circuit classes. *Log. Meth. Comput. Sci.*, 2(1–40), 2006.

22 A. Durand, A. Haak, and H. Vollmer. Model-theoretic characterization of Boolean and arithmetic circuit classes of small depth. In *Proc. LICS*, pages 354–363, 2018.

23 M.L. Furst, J.B. Saxe, and M. Sipser. Parity, circuits, and the polynomial-time hierarchy. In *Proc. FOCS*, pages 260–270, 1981.

24 Y. Gurevich and H. Lewis. A logic for constant-depth circuit. *Inf. Control*, 61:65–74, 1984. doi:10.1016/S0019-9958(84)80062-5.

25 W. Hesse, E. Allender, and D.A.M. Barrington. Uniform constant-depth threshold circuits for division and iterated multiplication. *Journal of Computer and System Sciences*, 65(4):695–716, 2002. doi:10.1016/S0022-0000(02)00025-9.

26 N. Immerman. Languages that capture complexity classes. *SIAM J. Comput.*, 16:760–778, 1987. doi:10.1137/0216051.

27 J. Johannsen. *A bounded arithmetic theory for constant depth threshold circuits*, volume 6 of *Springer Lecture Notes in Logic*, chapter GÖDEL '96, pages 224–234. Hájek, P., 1996.

28 D. Leivant. Predicative recurrence and computational complexity I: Word recurrence and poly-time. In *Feasible Mathematics*. Birkhüser, 1994.

29 D. Leivant and J.-Y. Marion. Ramified recurrence and computational complexity II: Substitution and poly-space. In *Proc. CSL*, pages 369–380, 1995.

30 D. Leivant and Y.-Y. Marion. Lambda calculus characterizations of poly-time. *Fundam. Inform.*, 19(1,2):167–184, 1993.

31 S. Lindell. A purely logical characterization of circuit uniformity. In *7th Structure in Complexity Theory Conf.*, pages 185–192, 1992.

32 A.A. Razborov. Lower bounds on the size of bounded depth circuits over a complete basis with logical addition. *Mathematical Notes of the Academy of Sciences of the USSR*, 41:333–338, 1987.

33 R. Smolensky. Algebraic methods in the theory of lower bounds for boolean circuit complexity. In Alfred V. Aho, editor, *Proceedings of the 19th Annual ACM Symposium on Theory of Computing, 1987, New York, New York, USA*, pages 77–82. ACM, 1987. doi:10.1145/28395.28404.

34 H. Vollmer. *Introduction to Circuit Complexity: A Uniform Approach*. Springer, 1999.